

ENME 408 Spring 2026

Project 4

Ethan Hardesty
Christopher Appiah

Honor code:

“I have neither received nor given aid to anyone outside my team during this project.”

A handwritten signature in black ink, appearing to read 'Ethan Hardesty', with a long, sweeping horizontal stroke extending to the right.A handwritten signature in black ink, appearing to read 'Chris Appiah', with a large, stylized 'C' and 'A'.

Problem Description

Consider a robot modeled as a point mass moving in a two-dimensional plane with position

$$q(t) = \begin{bmatrix} q_1(t) \\ q_2(t) \end{bmatrix} \in \mathbb{R}^2. \quad (1)$$

The robot acceleration in the inertial frame and the robot position are measured at discrete sampling instants

$$t_k = k\Delta t, \quad (2)$$

at a sampling frequency of 100 Hz, that is,

$$\Delta t = 0.01 \text{ s}. \quad (3)$$

The measured acceleration is

$$a_{m,k} = \begin{bmatrix} a_{1,m,k} \\ a_{2,m,k} \end{bmatrix}, \quad (4)$$

and the measured position is

$$q_{m,k} = \begin{bmatrix} q_{1,m,k} \\ q_{2,m,k} \end{bmatrix}. \quad (5)$$

Both measurements are corrupted by additive Gaussian noise and are provided in the file `circular_motion_noisy_position_acceleration.csv`

The provided CSV file has the following structure.

Column	Description
Time s	Time stamp t_k [s]
q1 true	True position $q_{1,k}$
q2 true	True position $q_{2,k}$
v1 true	True velocity $\dot{q}_{1,k}$
v2 true	True velocity $\dot{q}_{2,k}$
a1 true	True acceleration $a_{1,k}$
a2 true	True acceleration $a_{2,k}$
q1 meas	Noisy position measurement $q_{1,m,k}$
q2 meas	Noisy position measurement $q_{2,m,k}$
a1 meas	Noisy acceleration measurement $a_{1,m,k}$
a2 meas	Noisy acceleration measurement $a_{2,m,k}$

The columns labeled **true** are provided only for validation and comparison. They should not be used inside the Kalman filter implementation.

Your objective is to use these noisy acceleration and position measurements to estimate the full state

$$x_k \triangleq \begin{bmatrix} q_{1,k} \\ q_{2,k} \\ \dot{q}_{1,k} \\ \dot{q}_{2,k} \end{bmatrix}. \quad (6)$$

1. Measurement Visualization

Load the provided CSV file containing the noisy position and acceleration measurements.

- (a) Plot the noisy position measurements in the (q_1, q_2) plane.

Answer:

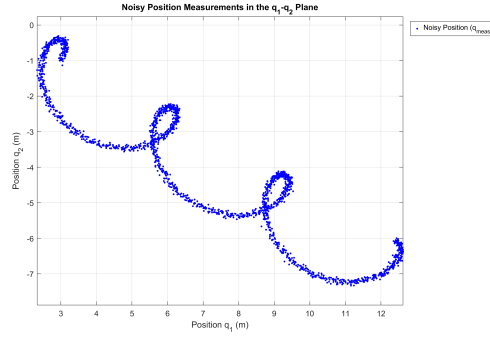


Figure 1: Two-dimensional spatial trajectory of noisy position measurements mapped across the q_1 - q_2 coordinate plane.

- (b) Plot the measured positions and accelerations versus time.

Answer:

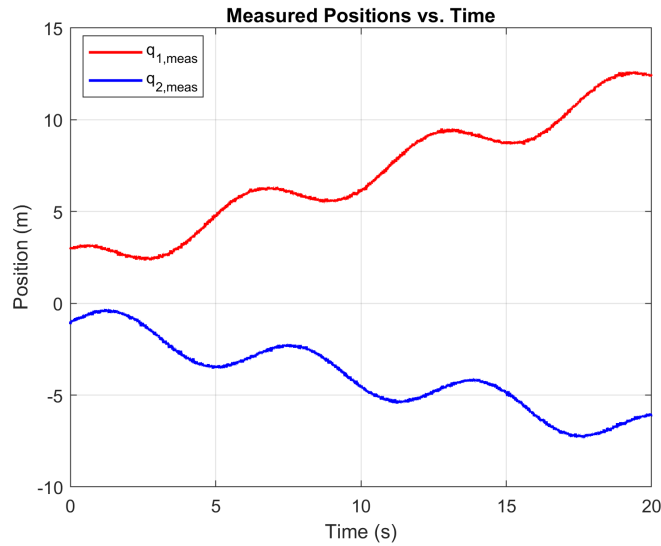


Figure 2: Time-domain measurement across execution intervals. This plot displays the linear positions over time.

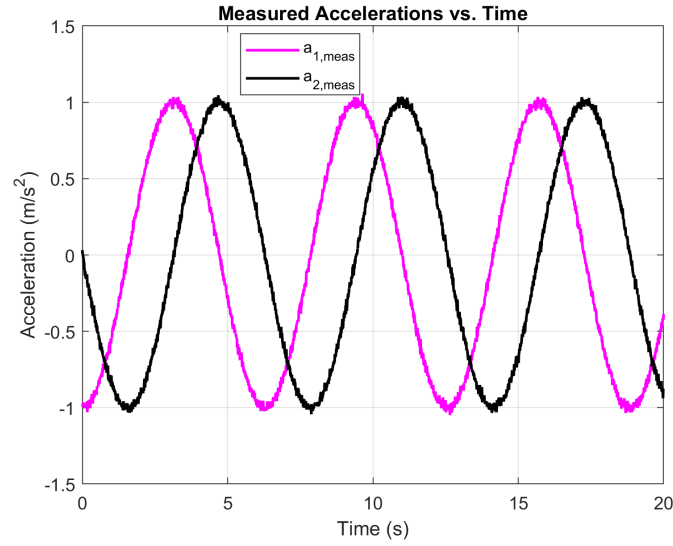


Figure 3: Time-domain measurement across execution intervals. This plot displays the measured system acceleration amplitudes relative to time.

- (c) Briefly comment on the quality of the measurements and the effect of measurement noise.

Answer: Because of measurement noise, the quality of the measurements isn't well-defined. The position measurements follow a structural circular motion, yet contain scattered data points.

2. **Discrete-time dynamics model** The translational dynamics are governed by

$$\ddot{q}(t) = a(t). \quad (7)$$

Define the state and input as

$$x(t) = \begin{bmatrix} q_1(t) \\ q_2(t) \\ \dot{q}_1(t) \\ \dot{q}_2(t) \end{bmatrix}, \quad u(t) = \begin{bmatrix} a_1(t) \\ a_2(t) \end{bmatrix}. \quad (8)$$

(a) Derive the continuous-time state-space model

$$\dot{x} = A_c x + B_c u. \quad (9)$$

Answer:

```
%Part (a)
% TODO: Define Ac and Bc.
Ac = [0, 0, 1, 0;
      0, 0, 0, 1;
      0, 0, 0, 0;
      0, 0, 0, 0];
Bc = [0, 0;
      0, 0;
      1, 0;
      0, 1];

% TODO: Define Cc and Dc only as needed to form the state-space model.
Cc = eye(4);
Dc = zeros(4,2);
```

Figure 4: Derivation of the continuous-time state-space model in MATLAB.

Given the translational dynamics $\ddot{q} = a(t)$, we assign individual variables to the components of the state vector

$$x(t) = \begin{bmatrix} q_1(t) \\ q_2(t) \\ \dot{q}_1(t) \\ \dot{q}_2(t) \end{bmatrix}$$

with dynamics

$$\dot{x}_1 = \dot{q}_1 = x_3$$

$$\dot{x}_2 = \dot{q}_2 = x_4$$

$$\dot{x}_3 = \ddot{q}_1 = a_1 = u_1$$

$$\dot{x}_4 = \ddot{q}_2 = a_2 = u_2$$

Expressing these linear differential equations in matrix form,

$$\dot{x} = A_c x + B_c u$$

yields

$$\begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \ddot{q}_1 \\ \ddot{q}_2 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} q_1 \\ q_2 \\ \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}$$

- (b) Use MATLAB `c2d()` with zero-order hold discretization and sampling time $\Delta t = 0.01$ s to determine the discrete-time model

$$x_{k+1} = A x_k + B u_k. \quad (10)$$

Answer:

```
%Part b
Ts = 0.01;

% TODO: Use ss() and c2d() with zero-order hold to compute A and B.
sysc = ss(Ac, Bc, Cc, Dc);
sysd = c2d(sysc, Ts, 'zoh');
A = sysd.A;
B = sysd.B;
```

Figure 5: Mapping of the continuous matrixes to discrete counterparts using MATLAB.

Under a Zero-Order Hold (ZOH) assumption with sampling time $\Delta t = 0.01$ s, the continuous matrices map to discrete counterparts,

$$x_{k+1} = A x_k + B u_k$$

via matrix exponentials:

$$A = e^{A_c \Delta t} = I + A_c \Delta t = \begin{bmatrix} 1 & 0 & 0.01 & 0 \\ 0 & 1 & 0 & 0.01 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$B = \left(\int_0^{\Delta t} e^{A_c \tau} d\tau \right) B_c = \begin{bmatrix} \frac{1}{2} \Delta t^2 & 0 \\ 0 & \frac{1}{2} \Delta t^2 \\ \Delta t & 0 \\ 0 & \Delta t \end{bmatrix} = \begin{bmatrix} 0.00005 & 0 \\ 0 & 0.00005 \\ 0.01 & 0 \\ 0 & 0.01 \end{bmatrix}$$

3. **Open-loop propagation using noisy acceleration.** Using the noisy acceleration measurements from the CSV file as the system input, propagate the discrete-time model forward in time without measurement correction.

- (a) Use the noisy acceleration measurements as input

$$u_k = a_{m,k}. \quad (11)$$

Answer:

```
% Part (a)
x_ol(:,k+1) = A * x_ol(:,k) + B * u_k;
end
```

Figure 6: Direct implementation of the discrete-time state space model.

- (b) Use the initial measured position and zero initial velocity.

Answer: The answer is observed part (c).

- (c) Plot the resulting trajectory and compare it with the noisy position measurements.

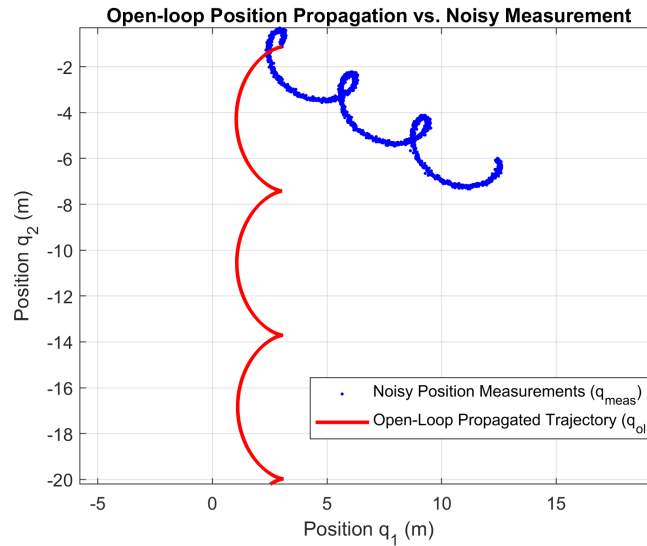


Figure 7: Open-loop Position Estimation of the Noisy Measurement

- (d) Explain why double integration of noisy acceleration causes drift in the position estimate.

Answer: The double integration of noisy acceleration drifts because the algorithm, depicted in Figure 6, maps raw measured accelerations. All sensor noise present in the acceleration measurements is integrated directly into the velocity calculations and integrated a second time into the position states. Consequently, this creates an exponential difference, resulting in the drift demonstrated in Figure 7.

4. **Kalman filter design.** Assume the available position measurements are

$$y_k = \begin{bmatrix} q_{1,m,k} \\ q_{2,m,k} \end{bmatrix} = Cx_k + v_k. \quad (12)$$

(a) Determine the output matrix C .

Answer:

```
% Part a
C = [1, 0, 0, 0;
     0, 1, 0, 0];
```

Figure 8: Output Matrix C

In the linear measurement equation

$$y_k = Cx_k + v_k,$$

expanding the matrices yields

$$\begin{bmatrix} q_{1,\text{meas}} \\ q_{2,\text{meas}} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}}_C \begin{bmatrix} q_1 \\ q_2 \\ v_1 \\ v_2 \end{bmatrix} + \begin{bmatrix} v_{1,k} \\ v_{2,k} \end{bmatrix}.$$

The first row of C , given by $[1, 0, 0, 0]$, maps the state vector to

$$1 \cdot q_1 + 0 \cdot q_2 + 0 \cdot v_1 + 0 \cdot v_2 = q_1.$$

Similarly, the second row of C , given by $[0, 1, 0, 0]$, maps the state vector to

$$0 \cdot q_1 + 1 \cdot q_2 + 0 \cdot v_1 + 0 \cdot v_2 = q_2.$$

Therefore, the measurement matrix C has dimensions 2×4 , which correctly maps a 4-element state vector to a 2-element measurement vector.

(b) Select reasonable process noise covariance Q and measurement noise covariance R . Justify your choice. Relate your choice of R to the measurement noise standard deviation observed in the data.

Answer:


```

% Calculate the error matrix between measured and true acceleration
acceleration_noise = a_meas - a_true;

% Compute the variance of each column (axis 1 and axis 2)
var_axes = var(acceleration_noise);

% Use the average variance across both axes as the scalar sigma_a2
sigma_a2 = mean(var_axes);

% Process covariance matrix and Measurement Noise Covariance Matrix,
% respectively
Q = B * (sigma_a2 * eye(2)) * B.';
R = 0.01*eye(2);

```

Figure 9: Process Noise Covariance Matrix and Measurement Noise Covariance Matrix

The process noise covariance matrix Q was selected based on the uncertainty injected into the state equations via the noisy input vector $u_k = a_{\text{meas}}, k$. Because the true and noisy measured accelerations are provided, the input noise characteristics can be explicitly calculated. Since the state-space model is driven by measured acceleration values rather than ideal inputs, the input noise maps directly onto the process dynamics. R is a 2x2 matrix because the system collects two independent position measurements (q_1 and q_2). Setting the diagonal elements to 0.01 indicates that the position sensor noise has a variance of $\sigma_q^2 = 0.01 \text{ m}^2$. The off-diagonal elements are exactly 0 because the sensor noise impacting the tracking axis q_1 is statistically uncorrelated with the noise impacting tracking axis q_2 .

- (c) Implement a discrete-time Kalman filter for state estimation.

Answer:

```

% TODO: Choose an initial covariance P and justify it.
P = diag([0.01, 0.01, 1.0, 1.0]);

```

Figure 10: The Estimate Covariance Matrix

The initial error covariance matrix is configured as a diagonal matrix matching the state vectors. The position entry variances are set to 0.01 because the initial state is initialized directly using the first physical measurement, matching the sensor's known variance. The velocity entry variances are set to higher values because the initial velocities are unmeasured guesses, indicating low confidence.

- (d) Clearly show the prediction step

$$\hat{x}_{k+1|k} = A\hat{x}_{k|k} + Bu_k \quad (13)$$

$$P_{k+1|k} = AP_{k|k}A^T + Q \quad (14)$$

and the measurement update step

$$K_{k+1} = P_{k+1|k}C^T (CP_{k+1|k}C^T + R)^{-1} \quad (15)$$

$$r_{k+1} = y_{k+1} - C\hat{x}_{k+1|k} \quad (16)$$

$$\hat{x}_{k+1|k+1} = \hat{x}_{k+1|k} + K_{k+1}r_{k+1} \quad (17)$$

$$P_{k+1|k+1} = (I - K_{k+1}C)P_{k+1|k} \quad (18)$$

A:

```

%% Prediction step
u_k = a_meas(k,:).';

% TODO: Implement prediction.
x_prior = A * xhat(:,k) + B * u_k;
P_prior = A * P * A.' + Q;



---


%% Measurement update step
y_kp1 = q_meas(k+1,:).';

% TODO: Compute Kalman gain.
S = C * P_prior * C.' + R;
K = P_prior * C.';

% TODO: Compute innovation/residual.
r = y_kp1 - C * x_prior;

% TODO: Compute posterior estimate and covariance.
x_post = x_prior + K * r;
P_post = (eye(4) - K * C) * P_prior;



---


%% Store posterior estimate
xhat(:,k+1) = x_post;
P = P_post;

```

Figure 11: Prediction and Measurement Step updates

x_{prior} and P_{prior} are the variables to calculate the predicted state and uncertainty projection forward in time by one sample interval ($\Delta t = 0.01$). To compute the Kalman filter, S , the innovation covariance matrix is needed because it is the difference between the actual measurement and the predicted measurement. x_{post} and P_{post} correct the state vector by scaling the residual with the Kalman Gain, and reduce the error covariance.

5. Results and discussion.

(a) Plot and compare the following trajectories:

- true trajectory
- noisy position measurements
- open-loop propagation using noisy acceleration
- Kalman filter estimate

Answer:

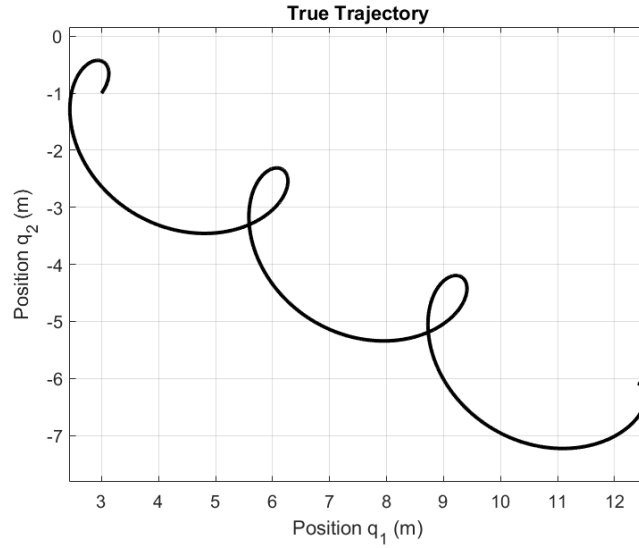


Figure 12: True Trajectory

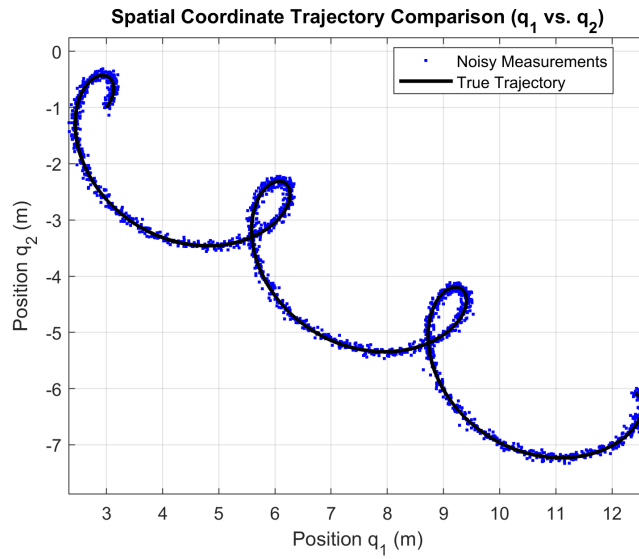


Figure 13: Noisy Position Measurements over True Trajectory

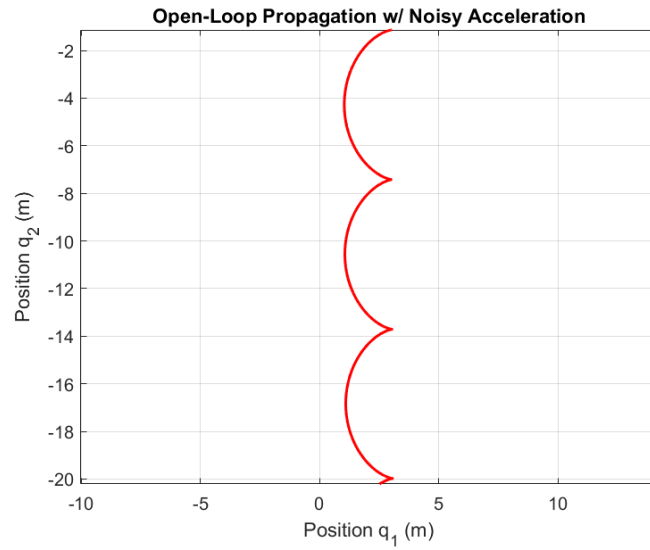


Figure 14: Open-Loop Propagation Using Noisy Acceleration

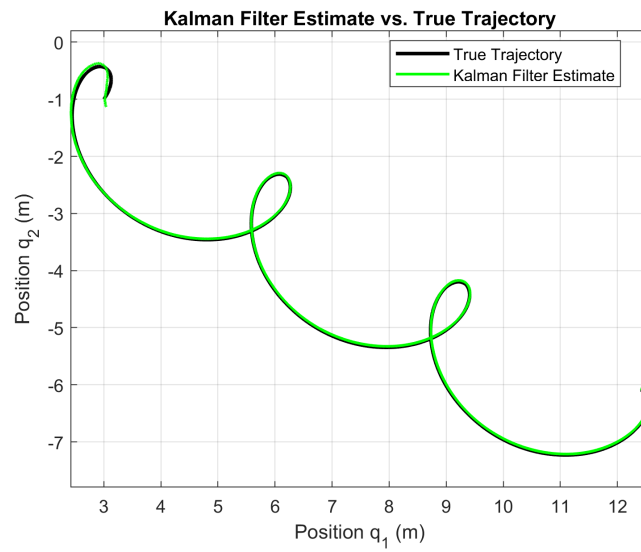


Figure 15: Kalman Filter Plotted Over True Trajectory

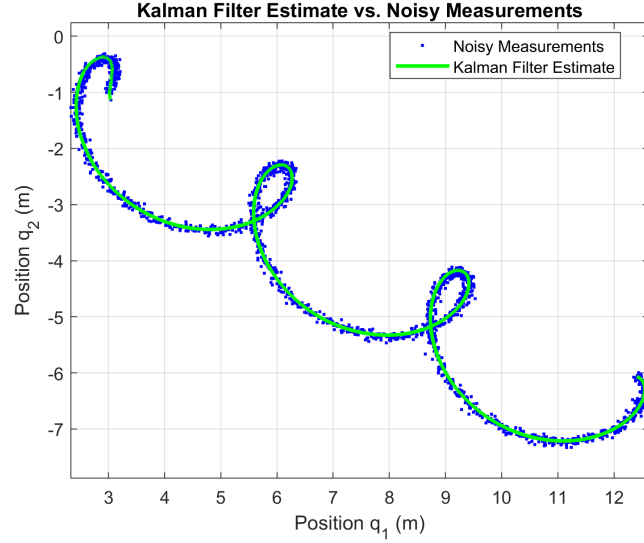


Figure 16: Kalman Filter Plotted Over Measurements

(b) Plot

$$q_1(t), \quad q_2(t), \quad \dot{q}_1(t), \quad \dot{q}_2(t) \quad (19)$$

versus time.

Answer:

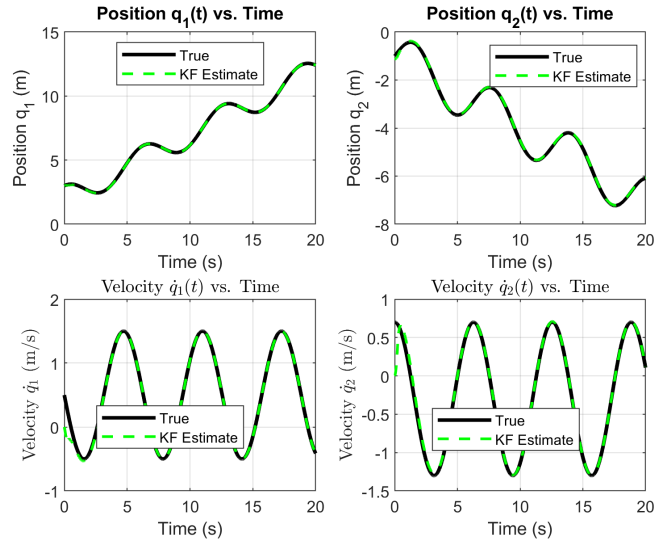


Figure 17: Time-domain tracking histories of the individual system states $q_1(t)$, $q_2(t)$, $\dot{q}_1(t)$, and $\dot{q}_2(t)$ evaluated over the entire duration. The discrete-time linear Kalman filter estimates demonstrate precise amplitude alignment and minimal phase lag when verified against the true system states.

(c) Discuss how the Kalman filter improves the estimate compared to direct acceleration integration.

A: While the estimate from the acceleration integration diverges from the true position, due to the lack of feedback correcting any error from the initial condition, the Kalman filter compares its estimate with the measurement data, and corrects itself when it diverges, allowing for a much more accurate estimate.

- (d) Discuss the effect of changing the covariance matrices Q and R .

A: The matrices Q and R rely on the system and its measurements, with Q depending on the accuracy and noise of the model and R depending on the accuracy and noise of the measurements or sensors. A higher Q matrix accounts for a more uncertain or noisy model, which increases the Kalman gain, K_{k+1} , and the system becomes more reliant on the measurements, which increase the response time to changes but leads to a higher sensitivity to measurement noise. In contrast, a lower Q accounts for a more robust and reliable model, which has a smoother estimate but has a slower response to changes and can drift from the true values if the model has any bias.

A high R matrix accounts for noisier measurements, so the response relies more on the system itself and predicting the value, which results in a smoother output but potentially inaccurate if the model has any bias. A low R matrix accounts for more accurate sensors, so the filter follows those measurements more closely, leading to a faster response but potentially magnifying measurement noise.

Credit Author Statement

Ethan Hardesty: Visualization, Software, Methodology

Christopher Appiah: Coding, Writing, Validation